

Supabase Integration – Copilot-Driven Setup Guide

This document explains **exactly** how the team must integrate Supabase with the Lovable-generated codebase using a **pre-written connection prompt** and **Copilot Agent inside VS Code**.

Important: **No one should manually write integration code**. All changes must be done by running the provided prompt in Copilot.

Prerequisites

Before starting, ensure the following are available:

- Codebase exported from **Lovable**
- Access to the **Supabase project** (URL + anon key)
- The **Connection Prompt** (provided separately in Notepad / text file)
- The **schema folder** (provided separately)
- VS Code with **Copilot Agent enabled**

Step-by-Step Integration Flow (MANDATORY ORDER)

STEP 1: Open the Lovable Codebase Locally

1. Clone or open the Lovable-exported repository in VS Code
2. Ensure the project builds and runs **before** Supabase integration

STEP 2: Add the `schema` Folder to the Codebase

1. Locate the provided `schema` folder (shared separately)
2. Copy the **entire `schema` folder**
3. Paste it into the **root of the codebase** (same level as `src`, `app`, or `pages`)

At this stage:

- No files should be edited
- No code should be written
- This step is only about **placing the schema folder**

STEP 3: Run the Connection Prompt via Copilot (CRITICAL STEP)

1. Open **Copilot Chat / Copilot Agent** in VS Code
2. Copy the **entire Connection Prompt** from the provided Notepad / text file
3. Paste it **as-is** into Copilot
4. Execute the prompt and allow Copilot to:
 - Scan the entire codebase
 - Read the `schema` folder as reference
 - Create or update Supabase client logic
 - Replace all static / mock / dummy data usage
 - Ensure all data flows from Supabase in real time

 Do NOT:

- Modify the prompt

- Add your own instructions
- Stop Copilot midway

STEP 4: Verify `.env` File Creation and Configuration

After Copilot finishes:

1. Check that a `.env` file now exists in the project root
2. Open `.env` and verify the following variables exist:

```
SUPABASE_URL=
SUPABASE_ANON_KEY=
```

3. Paste the correct values from the Supabase dashboard
4. Save the file

STEP 5: Rebuild and Run the Application

Run the following commands (depending on the stack):

```
npm install
npm run dev
```

STEP 6: Verify Real-Time Supabase Integration

Open the application in the browser (usually `http://localhost:8080`) and verify:

- Dashboard loads without errors
- Company listing is populated from Supabase
- Company cards use **short JSON data only**
- Clicking a company fetches **long JSON data** in real time
- No hardcoded, mock, or static data remains

If data changes in Supabase reflect immediately in the UI, the integration is successful.

Strict Rules for the Team

-  Do NOT manually write Supabase queries
-  Do NOT add joins or extra tables in code
-  Do NOT paste SQL schemas into Copilot
-  ONLY use the provided prompt
-  ONLY use the `schema` folder as reference
-  Let Copilot handle end-to-end integration

Expected Outcome

After completing these steps:

- The application is fully wired to Supabase
- All pages use **real-time backend data**
- Company cards use **short JSON**
- Company detail pages use **long JSON**
- The UI remains schema-faithful and scalable